Multi-agent system for smart machining, part II

Chapter I-d: exercises

Paolo Bosetti

2026-03-20

Table of contents

1 Exercise 1: PCA **1**
1.1 Example in R 1
1.2 Example in Python 8

2 Exercise 3: Analyze the MongoDB `MASSM_1` database **17**
2.1 Data preparation 17
2.2 Data query 20
2.2.1 Consideration about query speed 25

1 Exercise 1: PCA

1.1 Example in R

```
library(mongolite)
library(MASS)
library(tidyverse)
library(patchwork)
library(ggfortify)
```

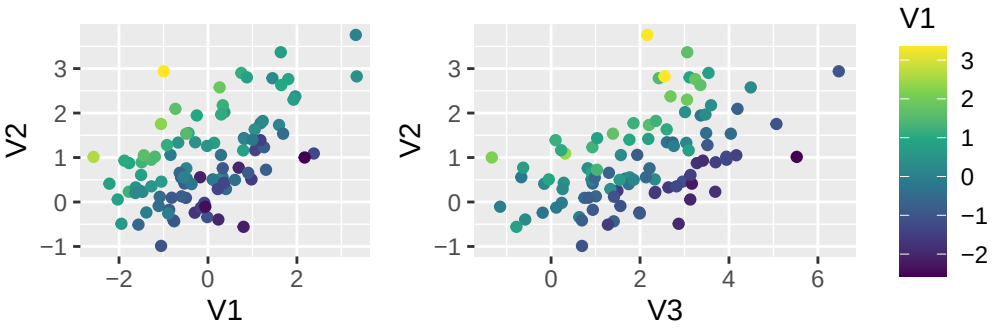
We can create a bivariate normal sample with a strong correlation between the two variables, and then apply PCA to it. The following code generates 100 samples from a bivariate normal distribution with mean zero and covariance matrix that has 2 and 1 on the diagonal and 0.8 on the off-diagonal, which means that the two variables are strongly correlated:

```r
set.seed(0)
cm <- matrix(c(
  2, 0.8, -0.9,
  0.8, 1, 0.7,
  -0.9, 0.7, 2.5
), nrow = 3, byrow = TRUE)
df <- mvrnorm(n = 100, mu = c(0, 1, 2), Sigma = cm) %>%
  as_tibble() %>%
  mutate(i = row_number(), .before = 1)

df %>% slice_head(n=5) %>% knitr::kable()
```

i	V1	V2	V3
1	-0.7317726	2.0944142	4.1921394
2	-0.5268653	0.4159005	0.9356861
3	-1.7793466	0.2307005	3.6913330
4	-1.1935736	1.0189383	3.9417735
5	-1.3851729	-0.2355991	1.9863782

On a plot:

```r
p1 <- df %>%
  ggplot(aes(x = V1, y = V2, color = V3)) +
  geom_point() +
  scale_color_viridis_c() +
  coord_equal() +
  theme(legend.position = "none")

p2 <- df %>%
  ggplot(aes(x = V3, y = V2, color = V1)) +
  geom_point() +
  scale_color_viridis_c() +
  coord_equal()
p1 + p2
```



The **actual**, sample covariance matrix is:

```
cov(df %>% select(-i)) %>% knitr::kable()
```

	V1	V2	V3
V1	1.4679204	0.6672781	-0.5239148
V2	0.6672781	0.9586985	0.7688910
V3	-0.5239148	0.7688910	2.2271363

Now we can apply PCA to this data. We can use the `prcomp` function, which performs PCA by computing the singular value decomposition of the centered and scaled data. The first principal component will capture the direction of maximum variance in the data, which should be along the line of correlation between the two variables.

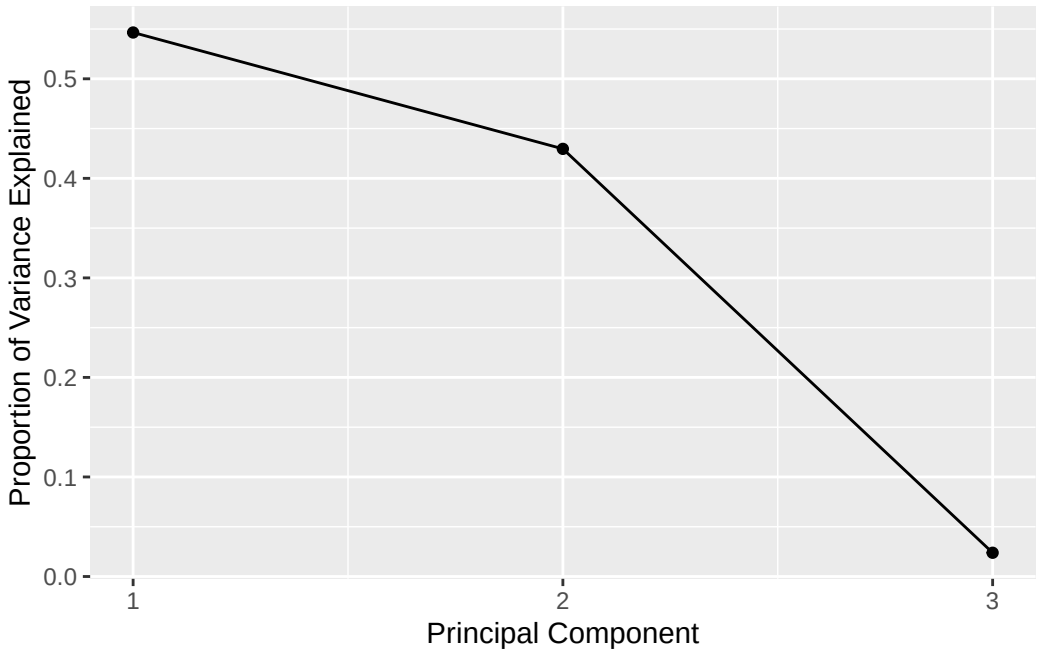
```
pca <- prcomp(df %>% select(-i), center = TRUE, scale. = TRUE)
summary(pca)
```

```
Importance of components:
                          PC1    PC2     PC3
Standard deviation     1.2805 1.1353 0.26752
Proportion of Variance 0.5465 0.4296 0.02386
Cumulative Proportion  0.5465 0.9761 1.00000
```

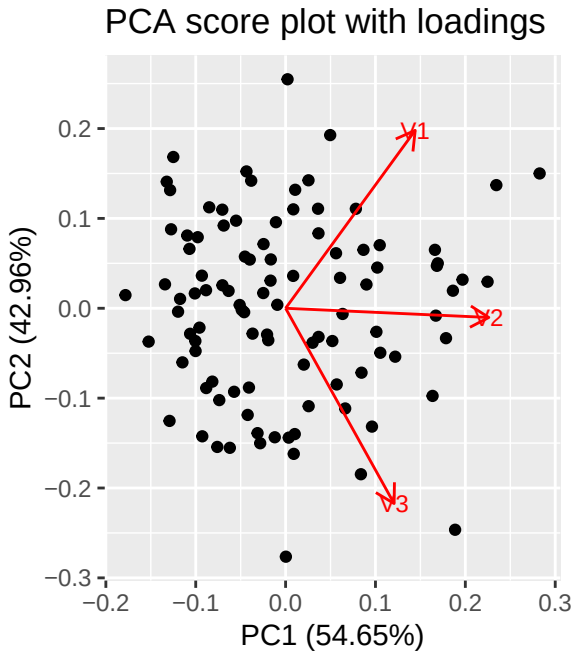
We can create the **scree plot** to visualize the proportion of variance explained by each principal component, in decreasing order:

```
tibble(
  a = 1:3,
  eta = summary(pca)$importance[2,]
) %>%
```

```
  ggplot(aes(x = a, y = eta)) +
  geom_line() +
  geom_point() +
  scale_x_continuous(breaks = 1:3) +
  labs(x = "Principal Component", y = "Proportion of Variance Explained")
```




```
autoplot(
  pca, data = df,
  loadings = TRUE,
  loadings.label = TRUE,
  loadings.label.size = 3
  ) +
  labs(title = "PCA score plot with loadings") +
  coord_equal()
```



So, in our example the first two principal components collect more than 97% of the total variance. We can thus reduce the dimensionality of the data from 3 to 2 by projecting the data onto the first two principal components. This is done by multiplying the centered and scaled data by the matrix of the first two eigenvectors (the loadings).

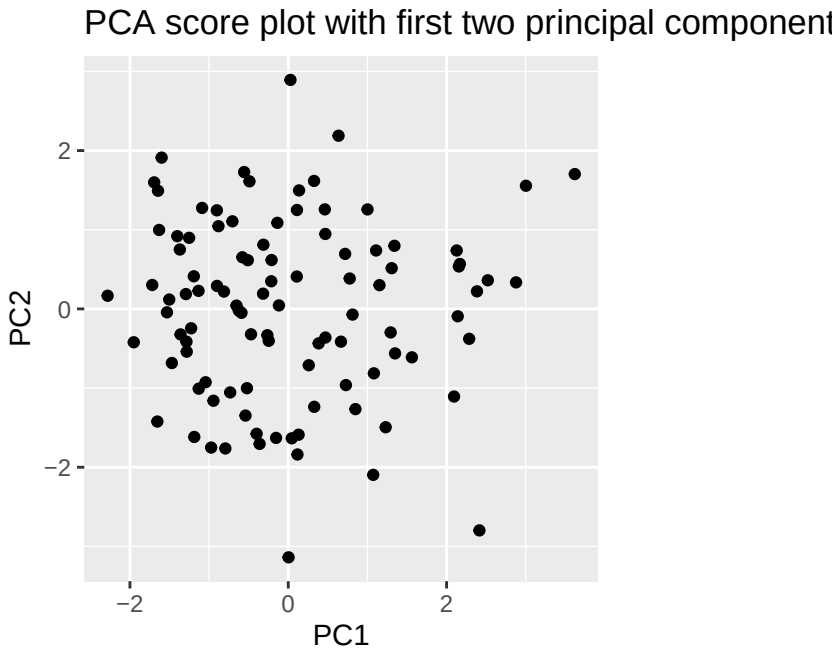
The **scores matrix** $\mathbf{T} = \mathbf{X}_c\mathbf{P}$ is:

```
df_T <- df %>%
  select(-i) %>%
  scale(center = TRUE, scale = TRUE) %>%
  tcrossprod(pca$rotation)
```

If we only take the first $a = 2$ principal components, we can create a new data frame with the scores of the first two principal components, $\mathbf{T_a} = \mathbf{X}_c\mathbf{P}_a$:

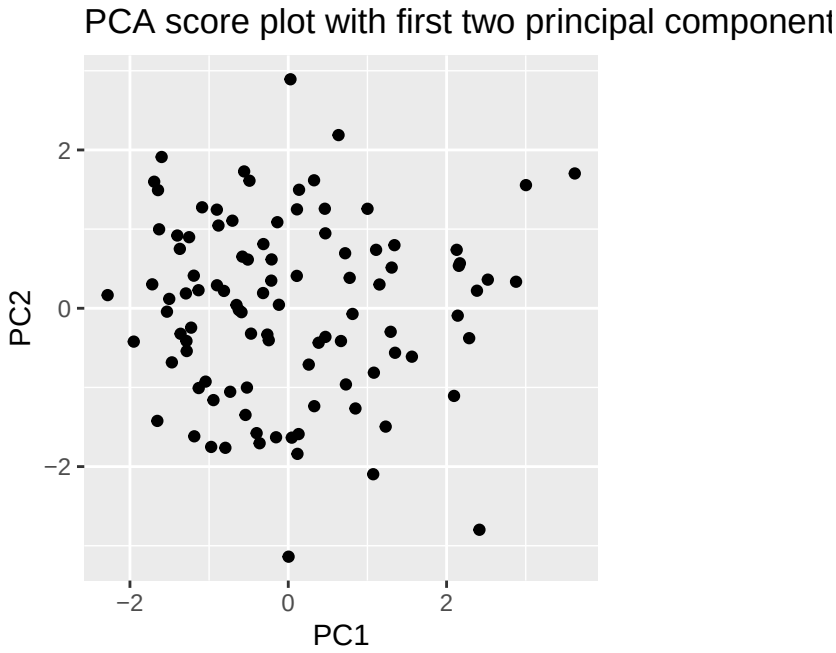
```
Xc <- df %>%
  select(-i) %>%
  scale(center = TRUE, scale = TRUE)

Ta <- (Xc %*%pca$rotation[,1:2]) %>%
  as_tibble()

Ta %>% ggplot(aes(x = PC1, y = PC2)) +
  geom_point() +
```

```
  coord_equal() +
  labs(title = "PCA score plot with first two principal components")
```



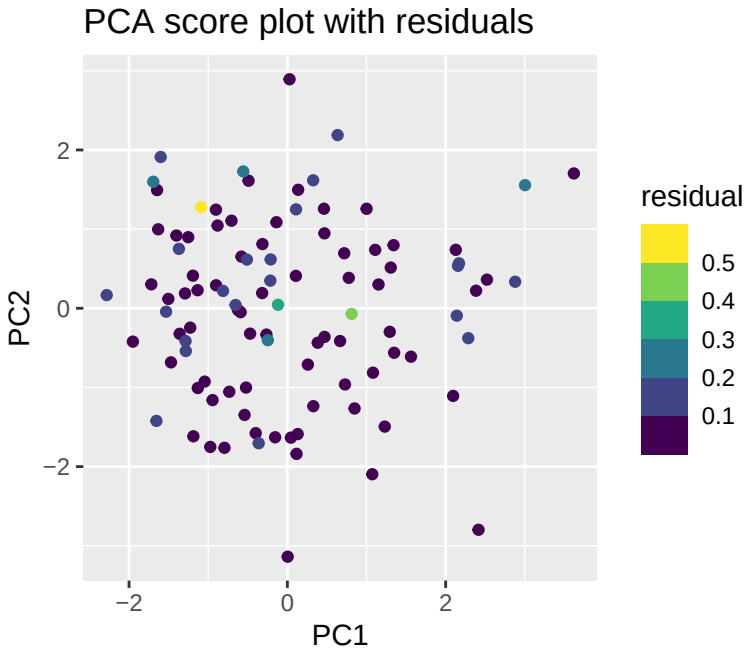
Or, equivalently, by directly tapping into the `x` component of the `pca` object, which contains the scores of all the principal components:

```
pca$x[,1:2] %>%
  as_tibble() %>%
  ggplot(aes(x = PC1, y = PC2)) +
  geom_point() +
  coord_equal() +
  labs(title = "PCA score plot with first two principal components")
```



Now we can calculate the residuals of the PCA reconstruction, which are the differences between the original data and the data reconstructed from the first two principal components. The reconstructed data can be obtained by multiplying the scores of the first two principal components by the transpose of the loadings of the first two principal components, and then adding back the mean of the original data.

```
df_hat <- tcrossprod(pca$x[, 1:2, drop = FALSE], pca$rotation[, 1:2, drop = FALSE])
epsi <- Xc - df_hat

pca$x[,1:2] %>%
  as_tibble() %>%
  mutate(residual = rowSums(epsi^2)) %>%
  ggplot(aes(x = PC1, y = PC2, color = residual)) +
  geom_point() +
  scale_color_viridis_b() +
  labs(title = "PCA score plot with residuals") +
  coord_equal()
```



1.2 Example in Python

The same workflow can be reproduced in Python with `numpy`, `pandas`, `scikit-learn`, and `matplotlib`.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(0)
cm = np.array([
    [2.0, 0.8, -0.9],
    [0.8, 1.0, 0.7],
    [-0.9, 0.7, 2.5],
])
mu = np.array([0.0, 1.0, 2.0])

X = rng.multivariate_normal(mean=mu, cov=cm, size=100)
df_py = pd.DataFrame(X, columns=["V1", "V2", "V3"])
df_py.insert(0, "i", np.arange(1, len(df_py) + 1))

df_py.head()
```

```
   i        V1        V2        V3
```

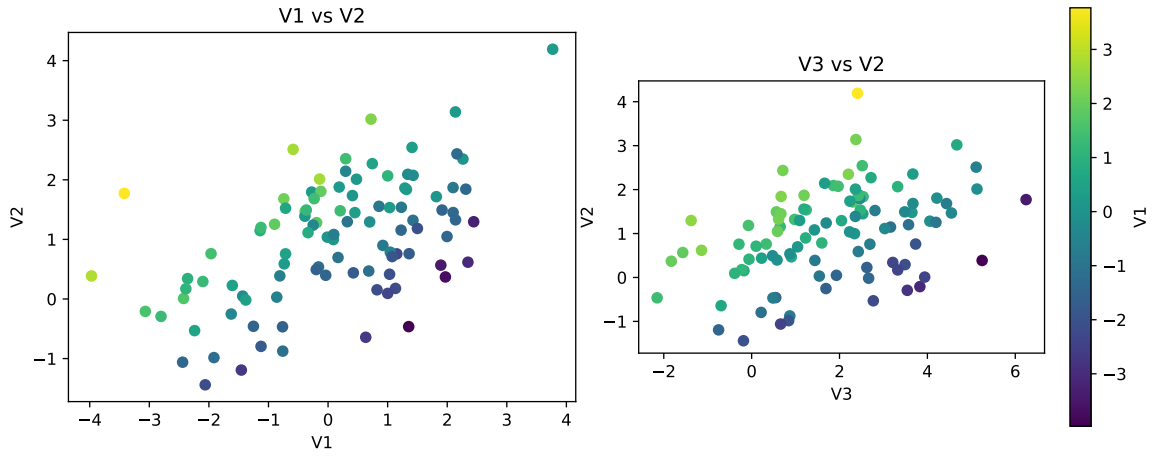
```
0  1  0.088701  0.997474  2.340826
1  2  0.450432  1.449486  2.529597
2  3 -2.357559  0.342007  3.212092
3  4  1.913295  1.492568  0.562541
4  5  2.444284  1.296232 -1.377366
```

We can visualize the same pairwise relationships as in the R example:

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(10, 4))

sc1 = axes[0].scatter(df_py["V1"], df_py["V2"], c=df_py["V3"], cmap="viridis")
axes[0].set_xlabel("V1")
axes[0].set_ylabel("V2")
axes[0].set_title("V1 vs V2")
axes[0].set_aspect("equal", adjustable="box")

sc2 = axes[1].scatter(df_py["V3"], df_py["V2"], c=df_py["V1"], cmap="viridis")
axes[1].set_xlabel("V3")
axes[1].set_ylabel("V2")
axes[1].set_title("V3 vs V2")
axes[1].set_aspect("equal", adjustable="box")
fig.colorbar(sc2, ax=axes[1], label="V1")
```

```
<matplotlib.colorbar.Colorbar object at 0x166b25250>
```

```
plt.tight_layout()
plt.show()
```



The sample covariance matrix is:

```
df_py.drop(columns="i").cov()
```

```
          V1        V2        V3
V1  2.258943  0.832087 -1.218430
V2  0.832087  1.037353  0.695967
V3 -1.218430  0.695967  2.856069
```

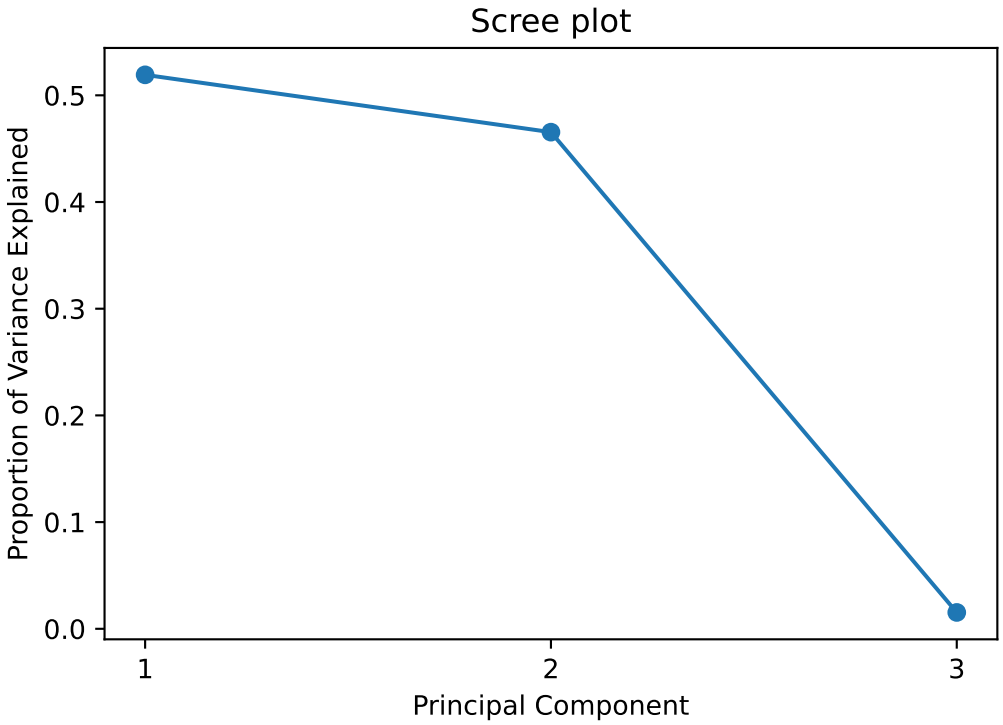
Now we standardize the variables and apply PCA:

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X_raw = df_py.drop(columns="i").to_numpy()
scaler = StandardScaler()
Xc = scaler.fit_transform(X_raw)

pca = PCA()
scores = pca.fit_transform(Xc)

summary_py = pd.DataFrame({
    "standard_deviation": np.sqrt(pca.explained_variance_),
    "proportion_of_variance": pca.explained_variance_ratio_,
    "cumulative_proportion": np.cumsum(pca.explained_variance_ratio_),
}, index=[f"PC{i}" for i in range(1, 4)])

summary_py
```

	standard_deviation	proportion_of_variance	cumulative_proportion
PC1	1.254315	0.519191	0.519191
PC2	1.187651	0.465470	0.984661
PC3	0.215596	0.015339	1.000000

The scree plot shows the variance explained by each principal component:

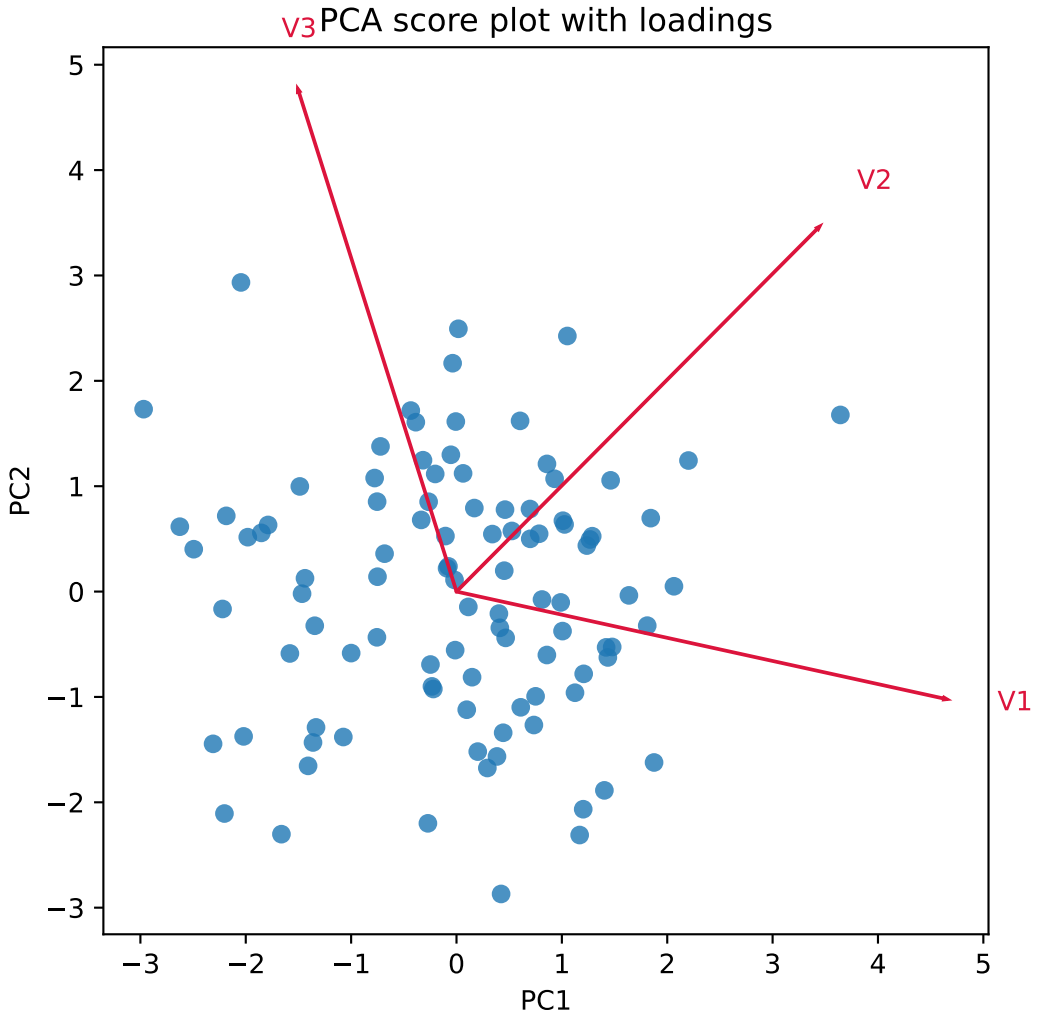
```
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot([1, 2, 3], pca.explained_variance_ratio_, marker="o")
ax.set_xticks([1, 2, 3])
ax.set_xlabel("Principal Component")
ax.set_ylabel("Proportion of Variance Explained")
ax.set_title("Scree plot")
plt.show()
```



We can also reproduce a PCA score plot with the loadings:

```
loadings = pca.components_.T
scores_df = pd.DataFrame(scores, columns=["PC1", "PC2", "PC3"])
```

```
fig, ax = plt.subplots(figsize=(6, 6))
ax.scatter(scores_df["PC1"], scores_df["PC2"], alpha=0.8)

arrow_scale = 6
for idx, feature in enumerate(["V1", "V2", "V3"]):
    ax.arrow(
        0, 0,
        loadings[idx, 0] * arrow_scale,
        loadings[idx, 1] * arrow_scale,
        color="crimson",
        width=0.01,
        length_includes_head=True
    )
    ax.text(
        loadings[idx, 0] * arrow_scale * 1.1,
        loadings[idx, 1] * arrow_scale * 1.1,
        feature,
        color="crimson"
    )

ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_title("PCA score plot with loadings")
ax.set_aspect("equal", adjustable="box")
plt.show()
```



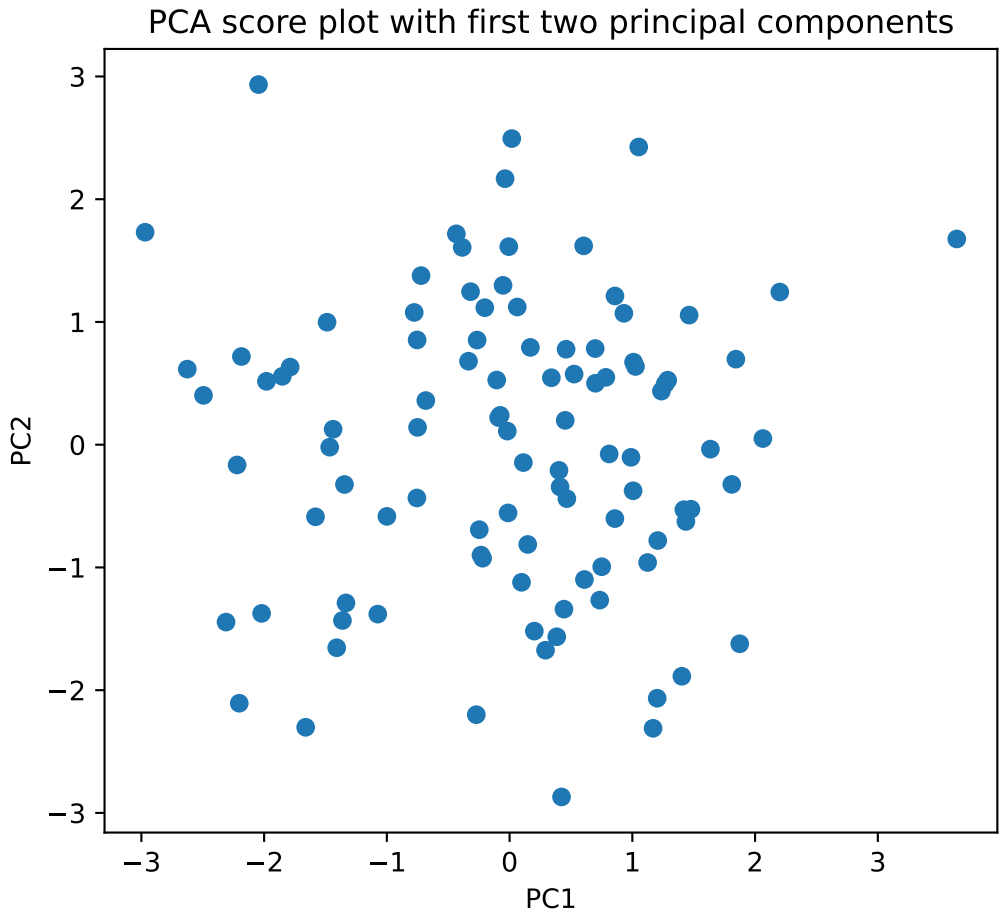
As in the R example, the first two principal components explain almost all of the variance, so we can project the data onto the first two columns of the loading matrix:

```
Ta = Xc @ loadings[:, :2]
Ta_df = pd.DataFrame(Ta, columns=["PC1", "PC2"])
Ta_df.head()
```

```
        PC1       PC2
0 -0.076512  0.239106
1  0.340248  0.545562
```

```
2 -1.851793  0.557004
3  1.420002 -0.529468
4  1.874924 -1.622015
```

```
fig, ax = plt.subplots(figsize=(6, 6))
ax.scatter(Ta_df["PC1"], Ta_df["PC2"])
ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_title("PCA score plot with first two principal components")
ax.set_aspect("equal", adjustable="box")
plt.show()
```



This is equivalent to taking the first two columns of the score matrix returned by `fit_transform`:

```
scores_df[["PC1", "PC2"]].head()
```

```
        PC1       PC2
0 -0.076512  0.239106
1  0.340248  0.545562
2 -1.851793  0.557004
3  1.420002 -0.529468
```

```
4  1.874924 -1.622015
```

Finally, we reconstruct the standardized data using only the first two principal components and compute the residuals:

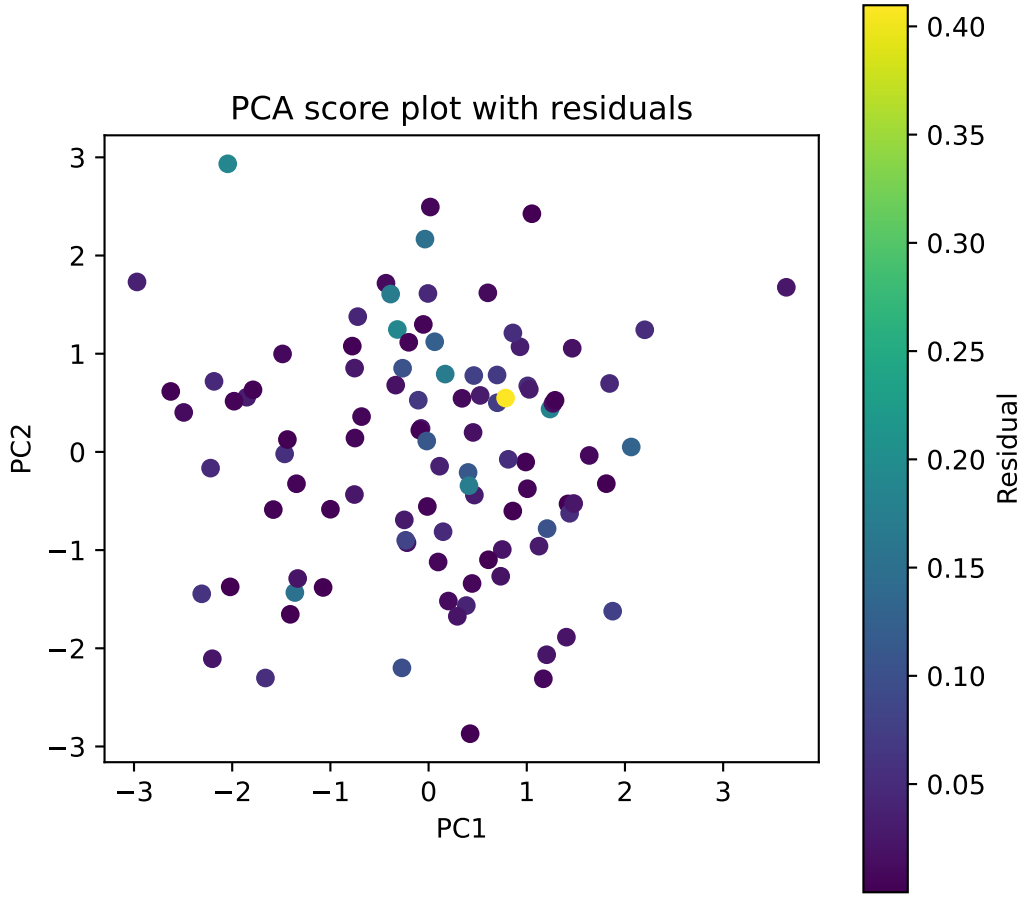
```
X_hat = scores[:, :2] @ loadings[:, :2].T
epsi = Xc - X_hat
residual = np.sum(epsi ** 2, axis=1)

scores_residual_df = scores_df[["PC1", "PC2"]].copy()
scores_residual_df["residual"] = residual
scores_residual_df.head()
```

```
        PC1       PC2  residual
0 -0.076512  0.239106  0.022793
1  0.340248  0.545562  0.010288
2 -1.851793  0.557004  0.032653
3  1.420002 -0.529468  0.000707
4  1.874924 -1.622015  0.078346
```

```
fig, ax = plt.subplots(figsize=(6, 6))
sc = ax.scatter(
    scores_residual_df["PC1"],
    scores_residual_df["PC2"],
    c=scores_residual_df["residual"],
    cmap="viridis"
)
ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_title("PCA score plot with residuals")
ax.set_aspect("equal", adjustable="box")
fig.colorbar(sc, ax=ax, label="Residual")
```

```
<matplotlib.colorbar.Colorbar object at 0x1729a1eb0>
```

```
plt.show()
```



2 Exercise 3: Analyze the MongoDB `MASSM_1` database

2.1 Data preparation

The `MASSM_1` database contains both the `acceleration` and the `force` collections. The `acceleration` collection contains the raw sensor data collected every 1 ms, each packing arrays of 10 elements corresponding to 10 kHz sampling rate. The `force` collection, conversely,

contains data sampled every 5 ms, and therefore contains only one sample every 5 documents in the `acceleration` collection.

> **Note**
>
> Force signals have been collected at a lower sampling rate because they are less dynamic than the acceleration signals, and therefore do not require a high sampling rate to capture their behavior. This is a common practice in data collection, where different types of signals are sampled at different rates based on their characteristics and the requirements of the analysis, aspecially after performing preliminary analysys via FFT (to define the minimum sampling frequency for each signal) and autocorrelation (to define the maximum sampling frequency based on the correlation time of the signal).

Before analyzing the data with PCA we have to perform a set of steps:

1. Unwind the four arrays `Ax`, `Ay`, `Az` and `microphone` in `measurements` so that we have one document for each element in the array. Keep all other fields as they are, and add a new field `measurements.idx` that is equal to the corresponding array index that originated the document.
2. Merge the force collection using the `message.timestamp` field as key. Carry over the last known value fields in force that are not present in this collection, merging the content of `measurements` from both collections.
3. Create a view that contains the merged data, and query it to load the data in R or Python for further analysis.

The corresponding pipelines are not trivial, but the AI assistant in MongoDB Compass can help you to write them. Let's start with the unwinding of the `acceleration` collection:

> **Prompt this:**
>
> Unwind the four arrays `Ax`, `Ay`, `Az` and `microphone` in `measurements` so that we have one document for each element in the array. Keep all other fields as they are, and add a new field `measurements.idx` that is equal to the corresponding array index that originated the document.

Save the result of this pipeline as a view `acceleration_exp`, and then don't forget to look at it and check that it contains the expected data. You should see that each document now contains a single value for `Ax`, `Ay`, `Az` and `microphone`, and a new field `measurements.idx` that indicates the index of the sample in the original array.

Next, we merge this view with the `force` collection, using the `message.timestamp` field as key. Again, liberally use the AI assistant in MongoDB Compass to write the aggregation pipeline for this merge, by giving it the following instruction:

Prompt this:

Merge the force collection using the `message.timestamp` field as key. Carry over the last known value fields in force that are not present in this collection, merging the content of `measurements` from both collections.

Before saving the result as view, open the aggregation pipeline and at the end add a new `$project` stage with the following content:

```
{
  _id: 0,
  timestamp: "$message.timestamp",
  Fx: "$message.measurements.Fx",
  Fy: "$message.measurements.Fy",
  Fz: "$message.measurements.Fz",
  Mz: "$message.measurements.Mz",
  sd_Fx: "$message.measurements.sd_Fx",
  sd_Fy: "$message.measurements.sd_Fy",
  sd_Fz: "$message.measurements.sd_Fz",
  sd_Mz: "$message.measurements.sd_Mz",
  Ax: "$message.measurements.Ax",
  Ay: "$message.measurements.Ay",
  Az: "$message.measurements.Az",
  microphone: "$message.measurements.microphone",
  idx: "$message.measurements.idx"
}
```

This way, we only keep the fields that are relevant for our analysis in a flat structure, easier and more simple to manage in R or Python.

Save the result as a new view called `all_data`. Now we can quesry this view to load the data in R or Python for further analysis. The view should contain all the fields from both collections, and you should be able to see the merged data when you query it.

Tip

Pipelines are just an array of *stages*, i.e. steps in building the desired view.
So, once you are content of the two views that you have built, you can just edit the first view `acceleration_exp` and add to the bottom the same list of stages that make the second view `all_data`. To do so, use the **`</> TEXT`** button in the toolbar of the pipeline editor, which allows you to edit the pipeline as a JSON array of stages. You can then copy and paste the stages from one pipeline to the other, and then save the view with the new pipeline.

2.2 Data query

Now that we have the `all_data` view, we can query it to load the data in R or Python for further analysis. For example, we can select the `Ax`, `Ay`, `Az`, `microphone`, and `Fx`, `Fy`, `Fz` and `Mz` fields, and create a data frame with these fields.

> Tip
>
> We also filter the data to avoid documents that do not contain both acceleration and force measurements.

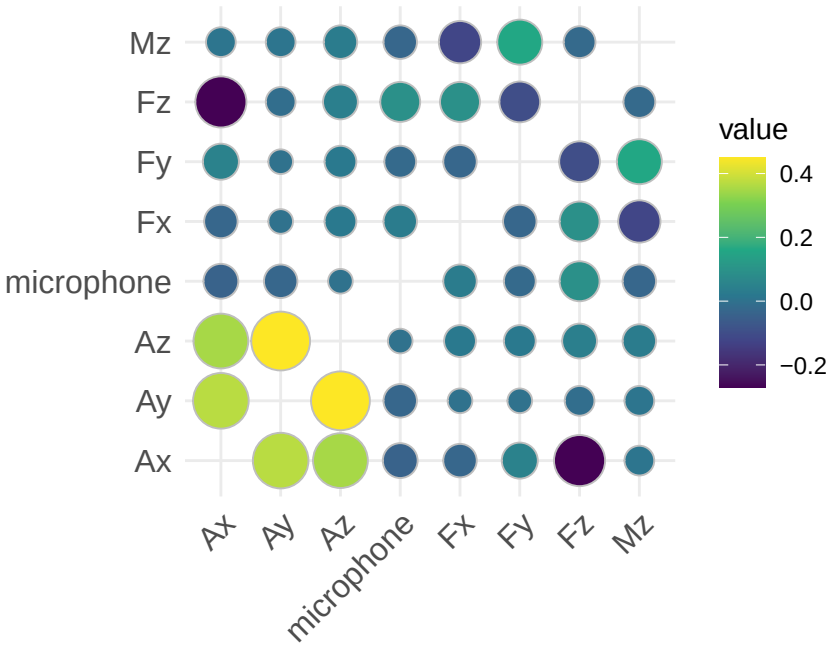
```
m <- mongo(
  collection = "all_data", db = "MASSM_1",
  url = "mongodb://localhost:27017")
query <- '{
  "Fx": { "$exists": true }
}'

df <- m$find(query=query, limit=10000)
```

Note that for PCA analysis we do not need the timestamp field, so we don't need to synthesize the time field from `timestamp` and `idx` fields.

To start, we calculate the correlation matrix and plot it on a heatmap to see the relationships between the different variables:

```
library(ggcorrplot)

cor_mat <- cor(df %>% select(Ax, Ay, Az, microphone, Fx, Fy, Fz, Mz))
ggcorrplot(cor_mat, method="circle", show.diag = FALSE, lab=FALSE) +
  scale_fill_viridis_c()
```

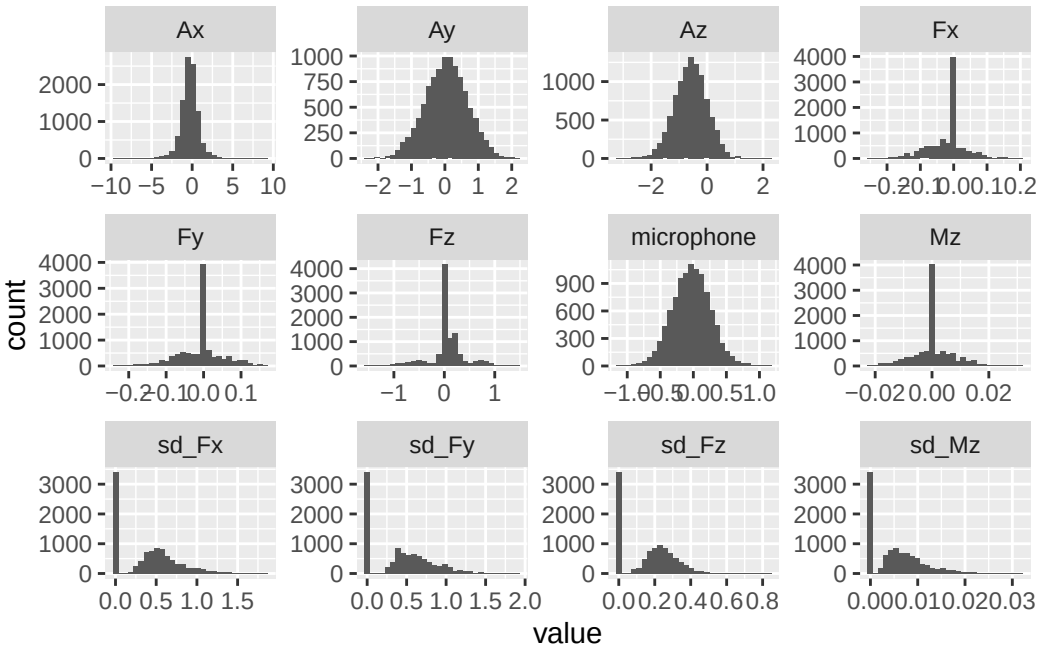
```
Scale for fill is already present.
Adding another scale for fill, which will replace the existing scale.
```



> **Tip**
>
> The `ggcorrplot` function (`install.packages("ggcorrplot")`) shows the size of the circles proportional to the absolute value of the correlation, and the color indicates the sign of the correlation (yellow for positive, violet for negative). We are removing the diagonal elements of the correlation matrix (which are all 1) to fully exploit the color scale.

Now we look at the distribution of each variable with a histogram:

```
df %>%
  pivot_longer(cols = -c(idx, timestamp)) %>%
  ggplot(aes(x = value)) +
  geom_histogram(bins = 30) +
  facet_wrap(~name, scales = "free")
```



Note that forces and moment have a strong peak arund zero, which corresponds to the idle state of the machine; also, as expected the standard deviations look as X^2 distributed, with a peak at zero and a long tail.

> **Todo**
>
> Repeat the analysis after filtering out the observations corresponding to idle state.

Finally, we can apply PCA to the data to see if we can identify some patterns in the data. We will use the `prcomp` function, which performs PCA by computing the singular value decomposition of the centered and scaled data. We will only use the `Ax`, `Ay`, `Az`, `microphone`, `Fx`, `Fy`, `Fz` and `Mz` fields for PCA, as they are the features of interest for our analysis.

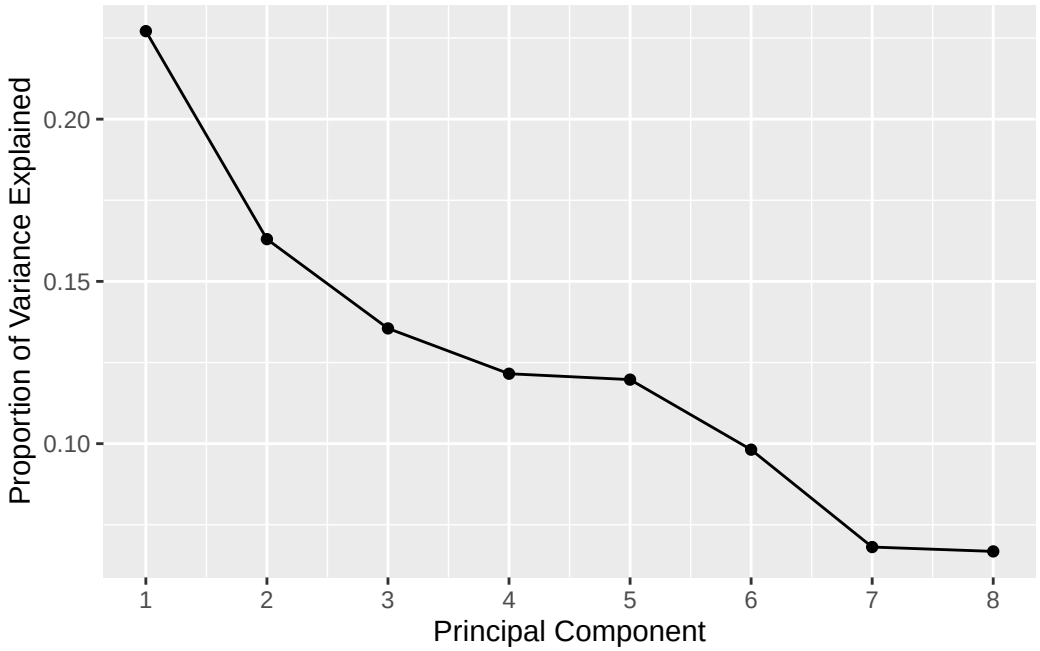
```
pca_fit <- prcomp(~Ax + Ay + Az + microphone + Fx + Fy + Fz + Mz, data=df, center = TRUE, sca
summary(pca_fit)
```

```
Importance of components:
                          PC1    PC2    PC3    PC4    PC5     PC6     PC7
Standard deviation     1.3480 1.1420 1.0412 0.9861 0.9787 0.88606 0.73828
Proportion of Variance 0.2271 0.1630 0.1355 0.1216 0.1197 0.09814 0.06813
Cumulative Proportion  0.2271 0.3901 0.5257 0.6472 0.7669 0.86508 0.93321
                           PC8
Standard deviation     0.73098
Proportion of Variance 0.06679
```

```
Cumulative Proportion  1.00000
```

It's probably better to look at the scree plot to decide how many principal components to keep for our analysis, rather than looking at the cumulative proportion of variance explained:

```
variance <- pca_fit$sdev^2 / sum(pca_fit$sdev^2)
variance %>%
  enframe(name = "PC", value = "variance") %>%
  ggplot(aes(x = PC, y = variance)) +
  geom_line() +
  geom_point() +
  scale_x_continuous(breaks = 1:8) +
  labs(x = "Principal Component", y = "Proportion of Variance Explained")
```



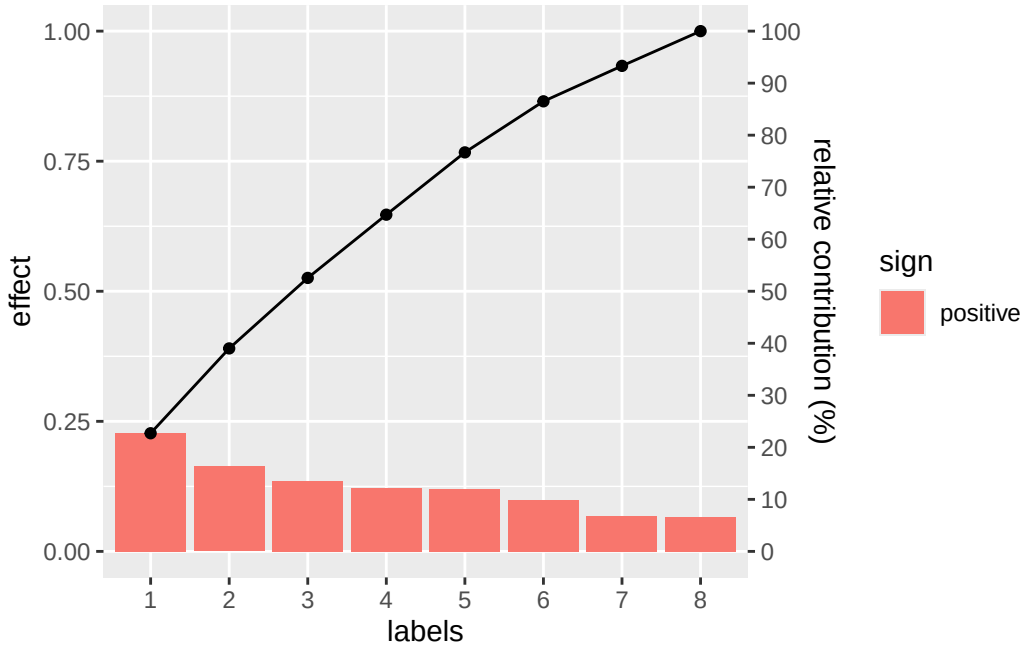
As an alternative, we can also create a Pareto chart, which is a bar plot of the variance explained by each principal component, sorted in decreasing order, with a line plot of the cumulative variance explained:

```
library(adas.utils)
```

```
Warning: package 'adas.utils' was built under R version 4.5.2
```

```
variance %>%
  enframe(name = "PC", value = "variance") %>%
  pareto_chart(labels=PC, values=variance)
```

```
Warning in pareto_chart.data.frame(., labels = PC, values = variance): 'pareto_chart.data.fra
Use 'geom_pareto' instead.
See help("Deprecated")
```



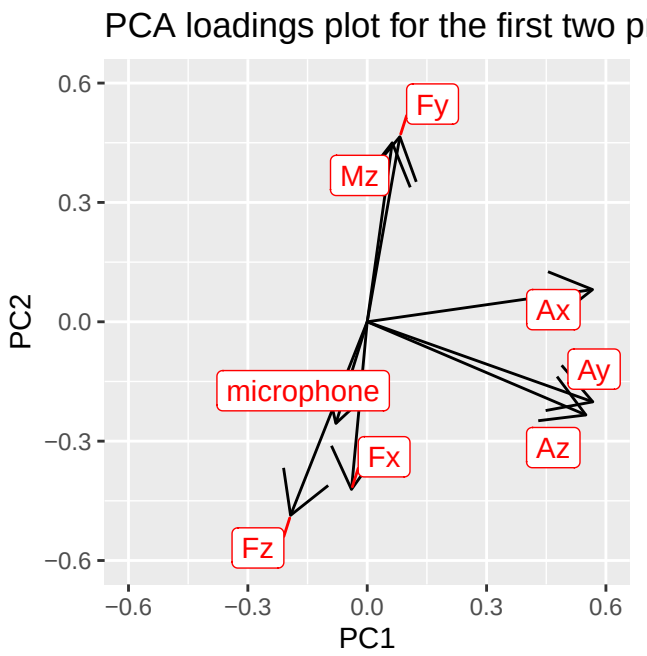
This shows that the first 6 principal components explain about 90% of the variance, so we can keep these 6 components for our analysis. We can also look at the loadings of the first two principal components to see which variables contribute the most to these components:

```
library(ggrepel)
```

```
Warning: package 'ggrepel' was built under R version 4.5.2
```

```
loadings <- pca_fit$rotation[, 1:2]
loadings %>%
  as_tibble(rownames = "variable") %>%
  ggplot(aes(x = PC1, y = PC2, label = variable)) +
  geom_segment(aes(x=0, y=0, xend = PC1, yend = PC2), arrow = arrow()) +
  geom_label_repel(color="red") +
```

```
labs(title = "PCA loadings plot for the first two principal components") +
coord_equal(xlim=c(-0.6, 0.6), ylim=c(-0.6, 0.6))
```



Todo

Using the PCA analysis, try to build explanatory statistics that allow you to identify the different operating states of the machine, and to understand which features are most relevant for this classification.

2.2.1 Consideration about query speed

If you try to make some more complex query on the `all_data` view, for example by filtering it by `timestamp`, you will see that it is quite slow, because it involves a lot of data and a complex pipeline.

On the other hand, if you want to build some running window statistics on that view, you need to repeatedly select a subset of the data,

You have two distinct options to speed up the queries:

1. save the view as a new collection, which will materialize the view and make the queries faster and — most importantly — will allow to create an index on the `timestamp` field, which is crucial for efficient queries on time series data. To do this, you can use the

`aggregate` method with the `$out` stage to save the result of the view as a new collection, and then create an index on the `timestamp` field of that collection (either from MongoDB Compass, or from R with `m$index(add = "timestamp")`).
2. since the view is already sorted according to time, you can just fetch data in chunkes using an **iterator**.

The first solution is very effective but has two main drawbacks:

- it doubles the disk space requirements, because you are actually duplicating your data in a new collection;
- you have to repeat the process every time you want to update the data, which can be a problem if you plan to collect and analyze data in real time.

So we now concentrate on the second solution, providing a basic example:

```
m <- mongo(
  collection = "all_data", db = "MASSM_1",
  url = "mongodb://localhost:27017")
query <- '{
  "Fx": { "$exists": true }
}'
it <- m$iterate(query=query, limit=10000)
```

Note the `iterate` method, which returns an iterator that allows you to fetch data in chunks, or *pages*. FOr example, suppose that we want to increamentally calculate the averages of all measurements in chunks of 10 observations; for the first 10 chuncks we can do:

```
averages <- tibble()
for (i in 1:10) {
  chunk <- it$page(10)
  averages <- bind_rows(averages, chunk %>% summarise(across(Fx:microphone, ~mean(., na.rm =
}
averages
```

```
# A tibble: 10 x 12
      Fx    Fy    Fz    Mz sd_Fx sd_Fy sd_Fz sd_Mz      Ax      Ay      Az
   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>   <dbl>   <dbl>   <dbl>
 1     0     0     0     0     0     0     0     0 -0.416   0.0574 -0.625
 2     0     0     0     0     0     0     0     0  0.104   0.343   0.0423
 3     0     0     0     0     0     0     0     0 -0.252   0.482  -0.624
 4     0     0     0     0     0     0     0     0 -0.947  -0.519  -1.10
 5     0     0     0     0     0     0     0     0 -0.189  -0.236  -0.529
```

```
 6     0     0     0     0     0     0     0     0  0.0347  0.519  -0.282
 7     0     0     0     0     0     0     0     0 -0.353   0.300  -0.623
 8     0     0     0     0     0     0     0     0 -0.707  -0.224  -1.00
 9     0     0     0     0     0     0     0     0 -0.423  -0.304  -0.646
10     0     0     0     0     0     0     0     0  0.514   0.485   0.0829
# i 1 more variable: microphone <dbl>
```